

CPS3830 Machine Learning Project 小组分工与工作流程手册

0. 项目一句话说明

我们这个 project 不是直接做完整 RAG 系统，而是把 RAG 检索阶段单独拿出来做一个机器学习比较实验。

我们要解决的问题是：

给定一个医疗问题和若干条候选医疗资料，不同 reranker 方法谁能把真正相关的资料排得更靠前？

最终我们会比较四种方法：

1. Embedding Similarity Baseline

原始向量相似度排序。

2. Open-source Reranker A

例如 BGE Reranker。

3. Open-source Reranker B

例如 Sentence-Transformers Cross-Encoder。

4. Our Lightweight ML Reranker

我们自己训练的传统机器学习模型，例如 Logistic Regression / Random Forest。

重点不是证明我们的方法一定最强，而是比较不同方法在医疗问答检索任务中的效果、速度、可解释性和不足。

1. 总体 workflow

整个项目按照下面这个流程走：

```
Step 1: Garry 新建 GitHub repo
    ↓
Step 2: Garry 上传统一数据文件 candidates.csv
    ↓
Step 3: 三个人同时开始跑各自模块
    ↓
Step 4: 每个人按照统一格式输出自己的 scores.csv
    ↓
Step 5: Garry 合并所有结果
    ↓
Step 6: 统一评估 Precision@K / MRR / latency
```

↓
Step 7: 整理 report、slides、README

最关键的原则：

所有人必须使用同一个 **candidates.csv**。

不要自己重新抽数据，不要自己改 qid，不要自己改 context_id。否则最后结果无法合并。

2. GitHub Repo 工作方式

2.1 Repo 名称建议

repo 名称：

CPS3830-Medical-Reranker

或者：

Medical-QA-Reranker-Comparison

2.2 分支方式

为了简单，建议不要搞复杂 Git workflow。

使用下面这种方式：

```
main
├── garry-ml
├── teammate-a-bge
└── teammate-b-crossencoder
```

每个人在自己的 branch 上工作。

- Garry: `garry-ml`
- 队友 A: `teammate-a-bge`
- 队友 B: `teammate-b-crossencoder`

如果不会用 branch，也可以更简单：

每个人只提交自己的文件夹，不要改别人文件。

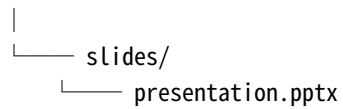
例如：

src/rerankers/bge_reranker.py	# 队友 A 负责
src/rerankers/cross_encoder.py	# 队友 B 负责
src/ml/train_lightweight_ml.py	# Garry 负责

3. Repo 目录结构

项目目录统一如下：

```
CPS3830-Medical-Reranker/
|
|—— README.md
|—— requirements.txt
|—— .gitignore
|
|—— data/
|   |—— raw/
|   |   |—— pediatric_sample.csv
|   |   |—— oncology_sample.csv
|   |
|   |—— processed/
|   |   |—— candidates.csv
|   |
|—— src/
|   |—— build_dataset.py
|   |—— embedding_baseline.py
|   |—— evaluate.py
|   |—— utils.py
|   |
|   |—— ml/
|   |   |—— feature_engineering.py
|   |   |—— train_lightweight_ml.py
|   |
|   |—— rerankers/
|   |   |—— bge_reranker.py
|   |   |—— cross_encoder_reranker.py
|   |
|—— results/
|   |—— embedding_baseline_scores.csv
|   |—— bge_scores.csv
|   |—— cross_encoder_scores.csv
|   |—— ml_scores.csv
|   |—— summary_metrics.csv
|   |
|—— report/
|   |—— proposal.md
|   |—— final_report.md
|   |—— figures/
```



```
graph TD; Root[ ] --- slides[slides/]; Root --- pptx[presentation.pptx];
```

每个文件夹的作用：

data/raw/

放原始或抽样后的医疗问答数据。

例如：

```
pediatric_sample.csv
oncology_sample.csv
```

大家一般不要直接改这里。

data/processed/

放统一处理后的数据。

最重要的是：

```
candidates.csv
```

这个文件是全组共同输入。

src/

放所有代码。

results/

放所有模型跑出来的结果。

每个人只负责生成自己的结果文件。

report/

放最终报告。

slides/

放展示 PPT。

4. 最核心文件：candidates.csv

4.1 这个文件是什么？

`candidates.csv` 是整个项目最重要的中间文件。

它的每一行表示：

一个医疗问题 + 一个候选医疗资料

也就是一个 question-context pair。

所有方法都基于它运行：

- Embedding baseline 读它；
- BGE reranker 读它；
- Cross-Encoder 读它；
- Lightweight ML reranker 读它；
- Evaluation script 也读它。

所以这个文件必须统一，不能每个人各做一份。

4.2 candidates.csv 必须包含的列

`data/processed/candidates.csv` 必须包含下面这些列：

```
qid
question
context_id
candidate_context
label
department
source_type
embedding_score
```

具体解释如下：

列名	类型	解释
qid	string	问题 ID，例如 q0001
question	string	医疗问题文本
context_id	string	候选资料 ID，例如 c000001
candidate_context	string	候选医疗资料文本

列名	类型	解释
label	int	1 表示相关，0 表示不相关
department	string	科室，例如 pediatrics / oncology
source_type	string	positive / random_negative / hard_negative
embedding_score	float	原始 embedding similarity 分数

4.3 candidates.csv 示例

```
qid,question,context_id,candidate_context,label,department,source_type,embedding_score
q0001,儿童发烧三天怎么办? ,c0001,儿童发烧期间应注意体温监测并观察精神状态...,
1,pediatrics,positive,0.8421
q0001,儿童发烧三天怎么办? ,c0002,儿童腹泻时应注意补液并观察脱水情况...,
0,pediatrics,hard_negative,0.7134
q0001,儿童发烧三天怎么办? ,c0003,肺癌化疗后可能出现恶心呕吐和免疫力下降...,
0,oncology,random_negative,0.3552
q0002,宫颈癌术后刺痛怎么办? ,c0021,宫颈癌术后若出现疼痛应注意伤口恢复和复查...,
1,oncology,positive,0.8233
```

4.4 label 是什么?

`label` 是标准答案，用来判断候选 context 到底是否相关。

- `label = 1` : 相关。
- `label = 0` : 不相关。

注意：

开源 reranker 自己打分时不需要 label。
但是最后评估它排得好不好，需要 label。

比如 BGE 把 10 条 context 排序后，我们要看：

它排在前三的 context 里面有几个 `label = 1`

这就是 Precision@3。

4.5 source_type 是什么?

`source_type` 表示这个样本是怎么来的。

可选值只能是下面三个：

```
positive
random_negative
hard_negative
```

含义：

source_type	解释
positive	原问题对应的医生回答，视为相关
random_negative	随机抽到的不同科室或明显不同疾病资料
hard_negative	同科室但不同疾病资料，更难区分

这个字段可以帮助我们后面分析模型错在哪里。

5. 每个人的具体分工

5.1 Garry：项目负责人 + 数据 + Lightweight ML Reranker

Garry 负责整个项目主线。

主要任务：

1. 新建 GitHub repo。
2. 上传项目结构。
3. 生成统一的 `candidates.csv`。
4. 跑 embedding baseline。
5. 训练 lightweight ML reranker。
6. 合并 A、B 的开源 reranker 结果。
7. 统一评估所有方法。
8. 检查最终 report 和 slides 的逻辑。

Garry 需要产出的文件：

```
data/processed/candidates.csv
results/embedding_baseline_scores.csv
results/ml_scores.csv
results/summary_metrics.csv
```

Garry 负责的代码文件：

```
src/build_dataset.py
src/embedding_baseline.py
```

```
src/ml/feature_engineering.py
src/ml/train_lightweight_ml.py
src/evaluate.py
```

5.2 队友 A: Open-source Reranker A

队友 A 负责接入第一个开源 reranker。

推荐模型：

BGE Reranker

队友 A 的任务：

1. 从 GitHub 拉取 repo。
2. 安装依赖。
3. 读取 `data/processed/candidates.csv`。
4. 对每一行 question-context pair 打分。
5. 对每个 qid 下的 candidate_context 按分数排序。
6. 输出统一格式的 `bge_scores.csv`。
7. 记录运行时间。
8. 在 report 中简单说明 BGE Reranker 的原理和使用方式。

队友 A 不能做的事：

- 不要自己重新抽数据。
- 不要自己改 `qid`。
- 不要自己改 `context_id`。
- 不要改 `candidates.csv` 的列名。
- 不要把输出文件改成自己的格式。

队友 A 需要产出的文件：

results/bge_scores.csv

队友 A 负责的代码文件：

src/rerankers/bge_reranker.py

5.3 队友 B: Open-source Reranker B + 报告辅助

队友 B 负责接入第二个开源 reranker。

推荐模型：

Sentence-Transformers Cross-Encoder

队友 B 的任务：

1. 从 GitHub 拉取 repo。
2. 安装依赖。
3. 读取 `data/processed/candidates.csv`。
4. 对每一行 question-context pair 打分。
5. 对每个 qid 下的 candidate_context 按分数排序。
6. 输出统一格式的 `cross_encoder_scores.csv`。
7. 记录运行时间。
8. 帮忙整理结果表格和图表。
9. 在 report 中写 Cross-Encoder 方法介绍和实验结果分析。

队友 B 不能做的事：

- 不要自己重新抽数据。
- 不要自己改 `qid`。
- 不要自己改 `context_id`。
- 不要改 `candidates.csv` 的列名。
- 不要把输出文件改成自己的格式。

队友 B 需要产出的文件：

`results/cross_encoder_scores.csv`

队友 B 负责的代码文件：

`src/rerankers/cross_encoder_reranker.py`

6. 所有模型输出格式必须统一

这是最重要的部分。

无论是 BGE、Cross-Encoder，还是我们的 ML reranker，输出都必须是同一种格式。

每个结果文件都必须包含这些列：

```
qid
context_id
score
rank
```

```
method
latency_ms
```

6.1 输出文件示例

bge_scores.csv

```
qid,context_id,score,rank,method,latency_ms
q0001,c0001,0.9123,1,bge,15.4
q0001,c0002,0.6431,2,bge,15.4
q0001,c0003,0.1022,3,bge,15.4
q0002,c0021,0.8876,1,bge,14.8
```

cross_encoder_scores.csv

```
qid,context_id,score,rank,method,latency_ms
q0001,c0001,0.8721,1,cross_encoder,22.6
q0001,c0003,0.2411,2,cross_encoder,22.6
q0001,c0002,0.1987,3,cross_encoder,22.6
q0002,c0021,0.9021,1,cross_encoder,23.1
```

ml_scores.csv

```
qid,context_id,score,rank,method,latency_ms
q0001,c0001,0.7612,1,lightweight_ml,2.1
q0001,c0002,0.4333,2,lightweight_ml,2.1
q0001,c0003,0.1208,3,lightweight_ml,2.1
q0002,c0021,0.7445,1,lightweight_ml,2.0
```

6.2 每一列是什么意思？

列名	类型	解释
qid	string	问题 ID，必须和 candidates.csv 一致
context_id	string	候选资料 ID，必须和 candidates.csv 一致
score	float	当前方法给出的相关性分数
rank	int	当前方法对同一个 qid 下 candidate 的排序，1 表示最相关
method	string	方法名称
latency_ms	float	该 qid 或该 pair 的运行时间，单位毫秒

6.3 method 名称必须统一

只能使用下面这些 method 名称：

```
embedding_baseline  
bge  
cross_encoder  
lightweight_ml
```

不要写成：

```
BGE-Reranker  
bge_reranker  
BGE_model  
crossencoder  
cross-encoder  
my_model  
random_forest
```

否则 evaluation script 合并时会很麻烦。

7. 每个人怎么单独运行自己的模块？

7.1 统一输入

所有人的输入都是：

```
data/processed/candidates.csv
```

不要直接读原始 CSV。

7.2 队友 A 的运行逻辑

队友 A 的代码逻辑是：

```
读取 candidates.csv  
↓  
对每一行 question + candidate_context 用 BGE 打分  
↓  
按 qid 分组  
↓  
每个 qid 内部按照 score 从高到低排序
```

```
↓  
生成 rank  
↓  
保存 results/bge_scores.csv
```

换句话说，BGE 不需要插入完整 RAG 本体。

它只是一个打分器：

```
bge(question, candidate_context) → score
```

7.3 队友 B 的运行逻辑

队友 B 的代码逻辑是：

```
读取 candidates.csv  
↓  
对每一行 question + candidate_context 用 Cross-Encoder 打分  
↓  
按 qid 分组  
↓  
每个 qid 内部按照 score 从高到低排序  
↓  
生成 rank  
↓  
保存 results/cross_encoder_scores.csv
```

Cross-Encoder 也不需要插入完整 RAG 本体。

它也是一个打分器：

```
cross_encoder(question, candidate_context) → score
```

7.4 Garry 的 lightweight ML 运行逻辑

Garry 的代码逻辑是：

```
读取 candidates.csv  
↓  
提取特征：  
    embedding_score  
    keyword_overlap
```

```
department_match
question_length
context_length
↓
使用 label 训练 ML model
↓
模型输出每个 question-context pair 的 relevance probability
↓
按 qid 分组排序
↓
保存 results/ml_scores.csv
```

Lightweight ML 和开源 reranker 最大区别是：

开源 reranker 直接读 question 和 context 原文。
Lightweight ML 读的是人工设计的数字特征。

8. 统一评估方式

最终由 `src/evaluate.py` 统一评估所有方法。

它会读取：

```
data/processed/candidates.csv
results/embedding_baseline_scores.csv
results/bge_scores.csv
results/cross_encoder_scores.csv
results/ml_scores.csv
```

然后合并 label，计算指标。

8.1 评估指标

我们最低需要计算：

```
Precision@1
Precision@3
MRR
Average latency
```

如果时间允许，再加：

Precision@5
NDCG@5
F1-score for lightweight ML

8.2 Precision@K 是什么？

例如某个问题有 10 个候选 context。

某个方法排出来前 3 个是：

rank	context_id	label
1	c0001	1
2	c0002	0
3	c0003	1

那么：

$\text{Precision@3} = \text{前 3 个里面相关的数量} / 3 = 2 / 3$

8.3 MRR 是什么？

MRR 看的是第一个相关答案排在第几位。

如果第一个 label = 1 的 context 排在第 1 位：

$\text{MRR} = 1 / 1 = 1$

如果第一个相关 context 排在第 3 位：

$\text{MRR} = 1 / 3 = 0.333$

越高越好。

8.4 最终 summary_metrics.csv 格式

最终结果表必须是：

```
method,precision_at_1,precision_at_3,mrr,avg_latency_ms
embedding_baseline,0.0000,0.0000,0.0000,0.0
bge,0.0000,0.0000,0.0000,0.0
cross_encoder,0.0000,0.0000,0.0000,0.0
lightweight_ml,0.0000,0.0000,0.0000,0.0
```

9. 文件命名规范

所有文件命名都用小写字母和下划线。

正确：

```
bge_scores.csv
cross_encoder_scores.csv
ml_scores.csv
summary_metrics.csv
train_lightweight_ml.py
```

错误：

```
BGE_result.csv
CrossEncoderFinal.csv
my_result_final_version_2.csv
ML_output_NEW.csv
```

不要出现空格，不要出现中文文件名，不要出现 final_final 这种版本名。

10. Git 提交规范

每次提交信息简单清楚。

推荐格式：

```
add bge reranker script
add cross encoder scores
update candidates dataset
add lightweight ml training
add evaluation metrics
update final report
```

不要写：

```
update
fix
aaa
new
final
```

11. 每个人今晚要完成什么？

Garry 今晚任务

1. 新建 GitHub repo。
2. 邀请两个队友加入 repo。
3. 建好目录结构。
4. 上传 README 初版。
5. 上传 `data/processed/candidates.csv` 初版。
6. 上传 `requirements.txt` 初版。
7. 上传输出格式说明。
8. 跑出 `embedding_baseline_scores.csv` 初版。

今晚 Garry 最重要的任务不是把 ML 模型训练到最好，而是先把统一输入文件和项目结构确定下来。

队友 A 今晚任务

1. 拉取 repo。
2. 创建自己的 branch。
3. 确认能读取 `data/processed/candidates.csv`。
4. 选择 BGE Reranker。
5. 用 5 到 10 行样例数据测试能否输出 score。
6. 确认输出格式符合：

```
qid,context_id,score,rank,method,latency_ms
```

今晚队友 A 不需要跑完整数据，先跑通小样例。

队友 B 今晚任务

1. 拉取 repo。
2. 创建自己的 branch。
3. 确认能读取 `data/processed/candidates.csv`。
4. 选择 Cross-Encoder。
5. 用 5 到 10 行样例数据测试能否输出 score。
6. 确认输出格式符合：


```
qid,context_id,score,rank,method,latency_ms
```

今晚队友 B 不需要跑完整数据，先跑通小样例。

12. 周一上午要完成什么？

Garry

1. 完成 feature engineering。
2. 训练 Logistic Regression。
3. 训练 Random Forest。
4. 选择表现更好的模型作为 lightweight ML reranker。
5. 输出：

```
results/ml_scores.csv
```

队友 A

1. 用完整 `candidates.csv` 跑 BGE Reranker。
2. 输出：

```
results/bge_scores.csv
```

1. 记录运行时间。
2. 写 BGE 方法说明。

队友 B

1. 用完整 `candidates.csv` 跑 Cross-Encoder。
2. 输出：

```
results/cross_encoder_scores.csv
```

1. 记录运行时间。
 2. 写 Cross-Encoder 方法说明。
-

13. 周一下午要完成什么？

Garry

1. 合并所有 scores。
2. 运行 `src/evaluate.py`。

3. 生成:

```
results/summary_metrics.csv
```

1. 检查是否有明显错误, 例如:
2. qid 对不上;
3. context_id 对不上;
4. rank 没按 qid 分组;
5. method 名称写错;
6. score 不是数字。

队友 A

1. 协助修复 BGE 结果格式。
2. 整理 BGE 的优缺点。
3. 帮忙写 result analysis。

队友 B

1. 协助修复 Cross-Encoder 结果格式。
2. 整理 Cross-Encoder 的优缺点。
3. 生成结果表格和图表。
4. 开始整理 slides。

14. 周一晚上要完成什么?

全组完成:

1. 最终检查 GitHub repo。
 2. 确保 README 说明如何运行。
 3. 确保 `results/summary_metrics.csv` 存在。
 4. 确保 report 写清楚:
 5. task 是什么;
 6. dataset 怎么构造;
 7. methods 怎么比较;
 8. evaluation metrics 是什么;
 9. results 怎么解释;
 10. limitations 是什么。
 11. 确保 slides 能讲清楚完整故事。
-

15. 最低可交版本

如果时间非常紧，最低可交版本必须包含：

1. `candidates.csv`
2. `embedding_baseline_scores.csv`
3. 一个开源 reranker 结果，例如 `bge_scores.csv`
4. `ml_scores.csv`
5. `summary_metrics.csv`
6. README
7. report 或 slides

如果两个开源 reranker 都跑出来，那项目更完整。

但如果其中一个开源 reranker 卡住，不要让整个项目停摆。至少保证：

embedding baseline vs one open-source reranker vs lightweight ML reranker

这个三方对比能完成。

16. 常见错误提醒

错误一：每个人自己做数据

不要这样做。

如果 A 用自己的 100 条数据，B 用自己的 200 条数据，Garry 用另一份数据，最后结果没有可比性。

所有人必须使用：

`data/processed/candidates.csv`

错误二：输出格式不统一

不要出现：

BGE 输出 `qid/context/score`
Cross-Encoder 输出 `question/context/rerank_score`
ML 输出 `id/cid/probability`

必须统一成：

```
qid,context_id,score,rank,method,latency_ms
```

错误三：rank 没有按 qid 分组

rank 必须在每个 question 内部单独排序。

正确：

```
q0001 的候选 context 排 rank 1-10  
q0002 的候选 context 也排 rank 1-10
```

错误：

```
整个 CSV 从 1 排到 1000
```

错误四：score 方向反了

所有方法都必须保证：

```
score 越高 = 越相关
```

如果某个模型输出的是 loss 或 distance，必须转换。

不要出现一个方法分数越低越好，另一个方法分数越高越好。

错误五：method 名字乱写

必须统一：

```
embedding_baseline  
bge  
cross_encoder  
lightweight_ml
```

错误六：开源 reranker 想接入完整 RAG

这一步暂时不需要。

我们的实验不是完整 RAG answer generation，而是单独评估 reranking。

开源 reranker 只需要做：

```
reranker(question, candidate_context) → score
```

不需要生成最终医疗回答。

17. 最终展示时怎么讲？

展示时按照这个逻辑讲：

1. 医疗问答中，retrieval quality 会影响最终回答质量。
2. 原始 embedding similarity 检索速度快，但可能不够精细。
3. 我们把 RAG 中的 context selection 问题抽出来，做成一个独立 ML reranking task。
4. 我们构造 question-context relevance dataset。
5. 我们比较四种方法：
 6. embedding baseline
 7. BGE reranker
 8. Cross-Encoder
 9. lightweight ML reranker
10. 我们用 Precision@K、MRR 和 latency 评估。
11. 我们分析不同方法的优缺点。
12. 我们的方法不一定最强，但轻量、可解释、容易结合医疗特征。
13. 未来可以扩展到完整 RAG 系统中，进一步测试它对最终回答质量的影响。

18. 一句话分工总结

Garry 负责：

```
数据、轻量 ML 模型、统一评估和最终整合。
```

队友 A 负责：

```
BGE Reranker，输出 bge_scores.csv。
```

队友 B 负责：

Cross-Encoder Reranker，输出 `cross_encoder_scores.csv`，并辅助报告和图表。

所有人共同遵守：

同一个 `candidates.csv` 输入。
同一种 `scores.csv` 输出格式。
同一个 `evaluation script` 评估。

只要这三点统一，大家就可以并行工作，最后结果也能顺利合并。